

Lab - gRPC

Objetivo

Demonstrar chamadas remotas com gRPC em Python, a partir de um contrato Protocol Buffers. A calculadora implementa soma, multiplicação, maior valor entre três números e divisão com quociente e resto.

Ambiente e execução

Execução em Linux usando uma imagem Python 3.12, com dependências fixadas. No computador Windows, Docker Desktop utiliza o backend WSL2. O README também descreve a execução direta em Linux com ambiente virtual. O servidor escuta nas interfaces do contêiner e a porta é publicada somente no endereço local do computador.

Dependências da aplicação e dos testes

```
grpcio==1.84.0
grpcio-tools==1.84.0
protobuf==7.36.2
pytest==9.1.1
pytest-timeout==2.4.0
iniconfig==2.3.0
packaging==26.3
pluggy==1.6.0
Pygments==2.21.0
setuptools==84.0.0
typing_extensions==4.16.0
```

Os comandos abaixo são executados na pasta do projeto.

A geração gRPC com Python requer o ambiente virtual com as dependências instaladas; o Docker também a executa durante o build.

```
docker compose up --build -d servidor
python -m grpc_tools.protoc -I. --python_out=. --grpc_python_out=. calculator.proto
docker compose exec -T servidor python calculator_client.py demo
docker compose exec -T servidor python calculator_client.py divide -17 5
docker compose exec -T servidor python calculator_client.py divide 17 0
docker compose run --rm testes
docker compose down
```

Implementação

O arquivo calculator.proto descreve as mensagens e os quatro procedimentos do serviço Calculator. Sum recebe a e b e retorna s; Multiply recebe a e b e retorna product; Maximum recebe a, b e c e retorna maximum; Divide recebe dividend e divisor e retorna quotient e remainder.

Os números associados aos campos da IDL são seus identificadores no protocolo, e não posições em uma lista. Os operandos e resultados usam double, com exceção do quociente, que usa int64. O contrato original de Sum foi preservado.

O compilador `grpc_tools.protoc` gera `calculator_pb2.py`, contendo as mensagens, e `calculator_pb2_grpc.py`, contendo o stub e as interfaces do serviço. Esses arquivos acompanham o projeto e não são editados manualmente. Calculator implementa os procedimentos; CalculatorStub realiza as chamadas remotas.

A divisão trunca o quociente em direção a zero e calcula resto = $\text{dividendo} - \text{quociente} \times \text{divisor}$. Assim, 17 dividido por 5 retorna quociente 3 e resto 2; -17 dividido por 5 retorna -3 e -2. Essa regra não combina $\text{int}(a/b)$ com $a \% b$, pois o operador % segue outra convenção para negativos.

Divisor zero, operandos não finitos, resultados infinitos e quocientes fora do intervalo de int64 são rejeitados com `INVALID_ARGUMENT`. O erro é devolvido ao cliente e o servidor permanece disponível. As operações com double estão sujeitas à precisão de ponto flutuante.

Os testes criam um servidor em porta livre, abrem um canal gRPC real e usam timeout nas chamadas. O arquivo correto do servidor é `calculator_server.py`; as menções a `calculator.py` no PDF são inconsistências editoriais. O script `demonstrar_geracao.py` reproduz a falha anterior à geração e o sucesso posterior numa cópia temporária.

Testes e resultados

A suíte cobre as quatro operações por chamadas gRPC reais. Verifica positivos, negativos, zeros, decimais, empates no máximo, quociente e resto, divisão por zero, NaN, infinitos, estouro numérico e continuidade do serviço após erros. Comparações de ponto flutuante usam tolerância.

Na validação registrada em 22/09/2026 às 17:41 UTC, 38 testes passaram em Linux, com Python 3.12.14, sem falhas, erros ou testes ignorados. O registro completo acompanha a entrega em `validacao.txt`.

Evidências de execução

Capturas reais realizadas no navegador em 23/09/2026 (horário de Brasília). Os terminais usam `ttyd` em um contêiner Linux temporário com Python 3.12.14. Os comandos Python são executados diretamente nesse contêiner; correspondem às chamadas `docker compose` descritas no README. Os horários dos logs e terminais estão em UTC. As interfaces foram geradas em `/tmp/interfaces` e comparadas aos módulos incluídos no projeto. O comando `cmp` terminou sem diferenças. Os testes de métodos selecionaram 28 casos; os casos especiais selecionaram 21, seguidos da suíte completa com 38 aprovados.

Figura 1

PG2 — Geração das interfaces gRPC

Execução real em Linux / Docker · terminal web ttyd · 23/09/2026

```
Linux | pg2_rpc | 2026-09-24 00:57:19 UTC

$ uname -s
Linux

$ python --version
Python 3.12.14

$ mkdir -p /tmp/interfaces

$ python -m grpc_tools.protoc -I. --python_out=/tmp/interfaces --grpc_python_out=/tmp/interfaces calculator.proto

$ ls -l /tmp/interfaces/calculator_pb2*.py
-rw-r--r-- 1 root root 2939 Sep 24 00:57 /tmp/interfaces/calculator_pb2.py
-rw-r--r-- 1 root root 8212 Sep 24 00:57 /tmp/interfaces/calculator_pb2_grpc.py

$ cmp calculator_pb2.py /tmp/interfaces/calculator_pb2.py && cmp calculator_pb2_grpc.py /tmp/interfaces/calculator_pb2_grpc.py

$ █
```

Geração das interfaces e mensagens gRPC no ambiente Linux

Figura 2

PG2 — Testes das quatro operações

Execução real em Linux / Docker · terminal web ttyd · 23/09/2026

```
calculator_integration_test.py::test_sum[256.5-128.8-385.3] PASSED [ 3%]
calculator_integration_test.py::test_sum[0-0-0] PASSED [ 7%]
calculator_integration_test.py::test_sum[-3--4--7] PASSED [ 10%]
calculator_integration_test.py::test_sum[-2-5-3] PASSED [ 14%]
calculator_integration_test.py::test_sum[0.1-0.2-0.3] PASSED [ 17%]
calculator_integration_test.py::test_multiply[6-7-42] PASSED [ 21%]
calculator_integration_test.py::test_multiply[0-20-0] PASSED [ 25%]
calculator_integration_test.py::test_multiply[-3-4--12] PASSED [ 28%]
calculator_integration_test.py::test_multiply[-2--5-10] PASSED [ 32%]
calculator_integration_test.py::test_multiply[2.5-1.2-3] PASSED [ 35%]
calculator_integration_test.py::test_maximum[9-2-3-9] PASSED [ 39%]
calculator_integration_test.py::test_maximum[1-9-3-9] PASSED [ 42%]
calculator_integration_test.py::test_maximum[1-2-9-9] PASSED [ 46%]
calculator_integration_test.py::test_maximum[5-5-2-5] PASSED [ 50%]
calculator_integration_test.py::test_maximum[-8--2--5--2] PASSED [ 53%]
calculator_integration_test.py::test_maximum[0-0-0-0] PASSED [ 57%]
calculator_integration_test.py::test_maximum[1.5-1.6-1.55-1.6] PASSED [ 60%]
calculator_integration_test.py::test_divide[20-5-4-0] PASSED [ 64%]
calculator_integration_test.py::test_divide[17-5-3-2] PASSED [ 67%]
calculator_integration_test.py::test_divide[0-5-0-0] PASSED [ 71%]
calculator_integration_test.py::test_divide[-17-5--3--2] PASSED [ 75%]
calculator_integration_test.py::test_divide[17--5--3-2] PASSED [ 78%]
calculator_integration_test.py::test_divide[-17--5-3--2] PASSED [ 82%]
calculator_integration_test.py::test_divide[7.5-2-3-1.5] PASSED [ 85%]
calculator_integration_test.py::test_divide[0.5-2-0-0.5] PASSED [ 89%]
calculator_integration_test.py::test_divide[-9223372036854775808-1--9223372036854775808-0] PASSED [ 92%]
calculator_integration_test.py::test_divide_by_zero[0] PASSED [ 96%]
calculator_integration_test.py::test_divide_by_zero[-0.0] PASSED [100%]
```

```
===== 28 passed, 10 deselected in 1.04s =====
```

```
$
```

Testes de soma, multiplicação, máximo e divisão

Figura 3

PG2 — Divisão, entradas inválidas e suíte completa

Execução real em Linux / Docker · terminal web ttyd · 23/09/2026

```
timeout func_only: False
collected 38 items / 17 deselected / 21 selected

calculator_integration_test.py::test_divide[20-5-4-0] PASSED [ 4%]
calculator_integration_test.py::test_divide[17-5-3-2] PASSED [ 9%]
calculator_integration_test.py::test_divide[0-5-0-0] PASSED [ 14%]
calculator_integration_test.py::test_divide[-17-5--3--2] PASSED [ 19%]
calculator_integration_test.py::test_divide[17--5--3-2] PASSED [ 23%]
calculator_integration_test.py::test_divide[-17--5-3--2] PASSED [ 28%]
calculator_integration_test.py::test_divide[7.5-2-3-1.5] PASSED [ 33%]
calculator_integration_test.py::test_divide[0.5-2-0-0.5] PASSED [ 38%]
calculator_integration_test.py::test_divide[-9223372036854775808-1--9223372036854775808-0] PASSED [ 42%]
calculator_integration_test.py::test_divide_by_zero[0] PASSED [ 47%]
calculator_integration_test.py::test_divide_by_zero[-0.0] PASSED [ 52%]
calculator_integration_test.py::test_invalid_numeric_values[Sum-SumRequest0] PASSED [ 57%]
calculator_integration_test.py::test_invalid_numeric_values[Sum-SumRequest1] PASSED [ 61%]
calculator_integration_test.py::test_invalid_numeric_values[Multiply-MultiplyRequest0] PASSED [ 66%]
calculator_integration_test.py::test_invalid_numeric_values[Multiply-MultiplyRequest1] PASSED [ 71%]
calculator_integration_test.py::test_invalid_numeric_values[Maximum-MaximumRequest] PASSED [ 76%]
calculator_integration_test.py::test_invalid_numeric_values[Divide-DivideRequest0] PASSED [ 80%]
calculator_integration_test.py::test_invalid_numeric_values[Divide-DivideRequest1] PASSED [ 85%]
calculator_integration_test.py::test_invalid_numeric_values[Divide-DivideRequest2] PASSED [ 90%]
calculator_integration_test.py::test_invalid_numeric_values[Divide-DivideRequest3] PASSED [ 95%]
calculator_integration_test.py::test_invalid_numeric_values[Divide-DivideRequest4] PASSED [100%]

===== 21 passed, 17 deselected in 1.05s =====

$ python -m pytest -q --color=yes
..... [100%]
38 passed in 0.85s

$
```

Testes de negativos, restos, divisor zero e valores inválidos

Figura 4

PG2 — Chamadas remotas pelo cliente

Execução real em Linux / Docker · terminal web ttyd · 23/09/2026

```
Linux | pg2_rpc | 2026-09-24 00:57:19 UTC

$ uname -s
Linux

$ python calculator_client.py demo
Sum(256.5, 128.8) = 385.3
Multiply(6, 7) = 42.0
Maximum(-8, -2, -5) = -2.0
Divide(17, 5): quociente=3, resto=2.0

$ python calculator_client.py divide -17 5
Divide(-17.0, 5.0): quociente=-3, resto=-2.0

$ python calculator_client.py divide 17 0
Erro INVALID_ARGUMENT: O divisor não pode ser zero.
[exit=1]

$ python calculator_client.py sum 1 2
Sum(1.0, 2.0) = 3.0

$ █
```

Chamadas do cliente ao servidor da calculadora

Referências

Enunciado da atividade: enunciado.pdf. Documentação oficial: <https://grpc.io/docs/languages/python/>
Procedimentos locais de execução: README.md.

Código fonte desenvolvido

As listagens seguintes contêm a implementação do laboratório e seus testes. As quebras de linhas longas são apenas de apresentação. Os arquivos executáveis originais acompanham a entrega.

Os módulos `calculator_pb2.py` e `calculator_pb2_grpc.py` são produzidos automaticamente pelo compilador. Estão incluídos no projeto; o contrato autoral completo é apresentado a seguir.

calculator.proto

```
syntax = "proto3";

message SumRequest {
  double a = 1;
  double b = 2;
}

message SumReply {
  double s = 1;
}

message MultiplyRequest {
  double a = 1;
  double b = 2;
}

message MultiplyReply {
  double product = 1;
}

message MaximumRequest {
  double a = 1;
  double b = 2;
  double c = 3;
}

message MaximumReply {
  double maximum = 1;
}

message DivideRequest {
  double dividend = 1;
  double divisor = 2;
}

message DivideReply {
  int64 quotient = 1;
  double remainder = 2;
}

service Calculator {
  rpc Sum (SumRequest) returns (SumReply);
  rpc Multiply (MultiplyRequest) returns (MultiplyReply);
  rpc Maximum (MaximumRequest) returns (MaximumReply);
  rpc Divide (DivideRequest) returns (DivideReply);
}
```

calculator_server.py

```
"""Calculadora gRPC derivada da implementação-base da disciplina."""

import argparse
import logging
```

```

import math
import platform
from concurrent import futures

import grpc

import calculator_pb2 as messages
from calculator_pb2_grpc import CalculatorServicer, add_CalculatorServicer_to_server

LOGGER = logging.getLogger("pg2")

def require_finite(context, *values):
    if not all(math.isfinite(value) for value in values):
        context.abort(grpc.StatusCode.INVALID_ARGUMENT, "Operandos e resultados devem ser finitos.")

class Calculator(CalculatorServicer):
    def Sum(self, request, context):
        require_finite(context, request.a, request.b)
        result = request.a + request.b
        require_finite(context, result)
        LOGGER.info("Sum(%s, %s) = %s", request.a, request.b, result)
        return messages.SumReply(s=result)

    def Multiply(self, request, context):
        require_finite(context, request.a, request.b)
        result = request.a * request.b
        require_finite(context, result)
        LOGGER.info("Multiply(%s, %s) = %s", request.a, request.b, result)
        return messages.MultiplyReply(product=result)

    def Maximum(self, request, context):
        require_finite(context, request.a, request.b, request.c)
        result = max(request.a, request.b, request.c)
        LOGGER.info("Maximum(%s, %s, %s) = %s", request.a, request.b, request.c, result)
        return messages.MaximumReply(maximum=result)

    def Divide(self, request, context):
        dividend, divisor = request.dividend, request.divisor
        require_finite(context, dividend, divisor)
        if divisor == 0:
            context.abort(grpc.StatusCode.INVALID_ARGUMENT, "O divisor não pode ser zero.")
        ratio = dividend / divisor
        require_finite(context, ratio)
        quotient = math.trunc(ratio)
        if not -(2**63) <= quotient < 2**63:
            context.abort(grpc.StatusCode.INVALID_ARGUMENT, "Quociente fora do intervalo int64.")
        remainder = dividend - quotient * divisor
        require_finite(context, remainder)
        LOGGER.info("Divide(%s, %s): quociente=%d resto=%s", dividend, divisor, quotient, remainder)
        return messages.DivideReply(quotient=quotient, remainder=remainder)

def create_server(host="127.0.0.1", port=50051):
    server = grpc.Server(futures.ThreadPoolExecutor(max_workers=10))
    add_CalculatorServicer_to_server(Calculator(), server)
    # Portas dinâmicas nos testes; endereço concreto do cliente, nunca [::].
    address = f"[{host}]:{port}" if ":" in host else f"{host}:{port}"
    bound_port = server.add_insecure_port(address)
    if not bound_port:
        raise OSError(f"Não foi possível escutar em {address}")
    return server, bound_port

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Servidor da calculadora gRPC")
    parser.add_argument("--host", default="127.0.0.1")
    parser.add_argument("--port", type=int, default=50051)
    args = parser.parse_args()

```

```

    logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s
%(message)s")
    server, port = create_server(args.host, args.port)
    server.start()
    LOGGER.info("Ambiente=%s Python=%s", platform.system(), platform.python_version())
    LOGGER.info("Calculadora gRPC em %s:%d", args.host, port)
    try:
        server.wait_for_termination()
    except KeyboardInterrupt:
        pass
    finally:
        server.stop(grace=2).wait()
        LOGGER.info("Servidor encerrado")

```

calculator_client.py

"""Cliente de terminal para as quatro operações da calculadora."""

```
import argparse
```

```
import grpc
```

```
import calculator_pb2 as messages
from calculator_pb2_grpc import CalculatorStub
```

```

def execute(stub, operation, values):
    if operation == "sum":
        a, b = values
        result = stub.Sum(messages.SumRequest(a=a, b=b), timeout=5)
        return f"Sum({a}, {b}) = {result.s}"
    if operation == "multiply":
        a, b = values
        result = stub.Multiply(messages.MultiplyRequest(a=a, b=b), timeout=5)
        return f"Multiply({a}, {b}) = {result.product}"
    if operation == "maximum":
        a, b, c = values
        result = stub.Maximum(messages.MaximumRequest(a=a, b=b, c=c), timeout=5)
        return f"Maximum({a}, {b}, {c}) = {result.maximum}"
    dividend, divisor = values
    result = stub.Divide(messages.DivideRequest(dividend=dividend, divisor=divisor),
timeout=5)
    return f"Divide({dividend}, {divisor}): quociente={result.quotient},
resto={result.remainder}"

```

```

def main():
    parser = argparse.ArgumentParser(description="Cliente da calculadora gRPC")
    parser.add_argument("--target", default="127.0.0.1:50051", help="endereço:porta do
servidor")
    commands = parser.add_subparsers(dest="operation", required=True)
    for name, count in (("sum", 2), ("multiply", 2), ("maximum", 3), ("divide", 2)):
        command = commands.add_parser(name)
        command.add_argument("values", type=float, nargs=count)
    commands.add_parser("demo", help="executar as quatro operações com entradas
conhecidas")
    args = parser.parse_args()
    examples = [
        ("sum", [256.5, 128.8]),
        ("multiply", [6, 7]),
        ("maximum", [-8, -2, -5]),
        ("divide", [17, 5]),
    ]
    if args.operation == "demo" else [(args.operation, args.values)]
    try:
        with grpc.insecure_channel(args.target) as channel:
            stub = CalculatorStub(channel)
            for operation, values in examples:
                print(execute(stub, operation, values))
        return 0
    except grpc.RpcError as error:
        print(f"Erro {error.code().name}: {error.details()}")
        return 1

```

```
if __name__ == "__main__":
    raise SystemExit(main())
```

calculator_integration_test.py

```
"""Testes da API por chamadas gRPC reais; servidor criado em porta livre."""
```

```
import sys
```

```
import grpc
import pytest
```

```
import calculator_pb2 as messages
from calculator_pb2_grpc import CalculatorStub
from calculator_server import create_server
```

```
@pytest.fixture(scope="module")
def grpc_server():
    server, port = create_server(port=0)
    server.start()
    yield server, port
    server.stop(grace=0).wait()
```

```
@pytest.fixture(scope="module")
def listen_address(grpc_server):
    return f"127.0.0.1:{grpc_server[1]}"
```

```
@pytest.fixture(scope="module")
def channel(listen_address):
    with grpc.insecure_channel(listen_address) as channel:
        grpc.channel_ready_future(channel).result(timeout=5)
    yield channel
```

```
@pytest.fixture(scope="module")
def calculator_client(channel):
    return CalculatorStub(channel)
```

```
@pytest.mark.parametrize("a,b,expected", [
    (256.5, 128.8, 385.3), (0, 0, 0), (-3, -4, -7), (-2, 5, 3),
    (0.1, 0.2, 0.3),
])
def test_sum(calculator_client, a, b, expected):
    result = calculator_client.Sum(messages.SumRequest(a=a, b=b), timeout=3)
    assert result.s == pytest.approx(expected)
```

```
@pytest.mark.parametrize("a,b,expected", [
    (6, 7, 42), (0, 20, 0), (-3, 4, -12), (-2, -5, 10), (2.5, 1.2, 3),
])
def test_multiply(calculator_client, a, b, expected):
    result = calculator_client.Multiply(messages.MultiplyRequest(a=a, b=b), timeout=3)
    assert result.product == pytest.approx(expected)
```

```
@pytest.mark.parametrize("a,b,c,expected", [
    (9, 2, 3, 9), (1, 9, 3, 9), (1, 2, 9, 9), (5, 5, 2, 5),
    (-8, -2, -5, -2), (0, 0, 0, 0), (1.5, 1.6, 1.55, 1.6),
])
def test_maximum(calculator_client, a, b, c, expected):
    result = calculator_client.Maximum(messages.MaximumRequest(a=a, b=b, c=c),
    timeout=3)
    assert result.maximum == pytest.approx(expected)
```

```
@pytest.mark.parametrize("dividend,divisor,quotient,remainder", [
    (20, 5, 4, 0), (17, 5, 3, 2), (0, 5, 0, 0),
    (-17, 5, -3, -2), (17, -5, -3, 2), (-17, -5, 3, -2),
    (7.5, 2, 3, 1.5), (0.5, 2, 0, 0.5), (-(2**63), 1, -(2**63), 0),
])
```

```

def test_divide(calculator_client, dividend, divisor, quotient, remainder):
    result = calculator_client.Divide(
        messages.DivideRequest(dividend=dividend, divisor=divisor), timeout=3
    )
    assert result.quotient == quotient
    assert result.remainder == pytest.approx(remainder)
    assert dividend == pytest.approx(result.quotient * divisor + result.remainder)

@pytest.mark.parametrize("divisor", [0, -0.0])
def test_divide_by_zero(calculator_client, divisor):
    with pytest.raises(grpc.RpcError) as error:
        calculator_client.Divide(messages.DivideRequest(dividend=17, divisor=divisor),
    timeout=3)
    assert error.value.code() == grpc.StatusCode.INVALID_ARGUMENT
    assert "zero" in error.value.details()

@pytest.mark.parametrize("method, rpc_request", [
    ("Sum", messages.SumRequest(a=float("inf"), b=1)),
    ("Sum", messages.SumRequest(a=sys.float_info.max, b=sys.float_info.max)),
    ("Multiply", messages.MultiplyRequest(a=1, b=float("nan"))),
    ("Multiply", messages.MultiplyRequest(a=sys.float_info.max, b=2)),
    ("Maximum", messages.MaximumRequest(a=1, b=2, c=float("-inf"))),
    ("Divide", messages.DivideRequest(dividend=float("nan"), divisor=2)),
    ("Divide", messages.DivideRequest(dividend=1, divisor=float("inf"))),
    ("Divide", messages.DivideRequest(dividend=1e308, divisor=1e-308)),
    ("Divide", messages.DivideRequest(dividend=2**63, divisor=1)),
    ("Divide", messages.DivideRequest(dividend=-(2**64), divisor=1)),
])
def test_invalid_numeric_values(calculator_client, method, rpc_request):
    with pytest.raises(grpc.RpcError) as error:
        getattr(calculator_client, method)(rpc_request, timeout=3)
    assert error.value.code() == grpc.StatusCode.INVALID_ARGUMENT
    # Um erro de argumento não deve encerrar o servidor.
    assert calculator_client.Sum(messages.SumRequest(a=1, b=2), timeout=3).s == 3

```